

The implementation of DevOps techniques allows you to automate and optimise processes using technology, but it all starts with the culture within the organisation and the people who work in it. To instil a DevOps culture, you need to radically change the way employees approach work and collaborate. If an organisation adheres to the DevOps culture, it can create all the conditions for the development of high performance teams.

DevOps techniques

In addition to implementing the DevOps culture, teams implement the DevOps approach by applying specific techniques throughout the application life cycle. Some of these techniques help speed up, automate and improve the execution of a particular stage. Others cover multiple stages, helping teams create holistic processes that help increase productivity.

Continuous integration and delivery (CI/CD): Continuous integration is a software development technique in which developers often merge code changes into a master code branch. Continuous integration uses automatic testing, which is performed every time new code is committed, so the code in the main branch is always stable.

Continuous delivery is the regular automatic deployment of new versions of applications in the production environment. By automating the actions required for deployment, teams reduce the likelihood of problems during deployment and create conditions for more frequent updates.

The result of both techniques is the CI/CD process, which includes full automation of all actions between code commit and deployment in the production environment. The implementation of CI/CD allows teams to focus on creating code, reduce costs, and reduce the human impact of performing routine steps. CI/CD also allows you to speed up the process of

deploying new code and reduce the risks associated with it. In this case, deployments are performed more often and in smaller increments. This allows teams to increase flexibility and productivity, and also gives them more confidence in the correctness of the code being executed.

Version control: Version control is a technique for working with code in versions, that is, tracking fixes and change history to simplify code analysis and recovery. This technique is usually implemented using version control systems such as Git, which allow multiple developers to work together to create code. These systems provide a clear process for combining code changes in the same files, resolving conflicts and rolling back changes to earlier states.

The use of version control is a fundamental DevOps practice that helps development teams interact, share code writing tasks among team members, and save all code for quick recovery if necessary.

Versioning is also a necessary element in other methodologies, such as continuous integration and infrastructure as code.

Flexible software development: Agile is an approach to software development that focuses on team collaboration, customer and user feedback, and high adaptability to change over short release cycles. Teams that use a flexible methodology constantly provide customers with changes and improvements, collect their feedback, take into account their wishes and needs, and then adapt to these. The flexible methodology differs significantly from other more traditional systems, such as the cascade methodology, which provides for long release cycles with successive stages. Two popular concepts — Kanban and Scrum — are associated with the agile methodology.

Who is a DevOps engineer?

Development operations or DevOps is a step towards optimising interactions in small teams to increase the speed of application production. The idea is to strengthen the development team with the knowledge of procedures and approaches, in order to manage the product environment. In other words, developers must understand and know how their product works in certain conditions, must understand how to deploy their product, and know what characteristics of the environment to tweak to improve performance. So, for some time, there were developers who took a DevOps approach. They wrote build and package scripts to simplify their operations and make the production environment work.

However, the complexity of the solution architecture and the mutual influence of the infrastructure components over time began to worsen the performance of the environments. With each iteration, more and more deep understanding of certain components was required, reducing the productivity of the developer due to the additional costs of understanding the components and tuning the systems for a specific task. The developer's own cost grew and so did the cost of the product along with it. The requirements for new developers in the team jumped sharply, because they also had to cover the responsibilities of the 'star' of development and, of course, the 'stars' became less and less available. It is also worth noting that few developers are interested in the specifics of packet processing by the operating system kernel, packet routing rules and host security aspects. The logical step was to attract an administrator who was familiar with this and assign this format of responsibilities to him or her, which, thanks to their experience, allowed the team to achieve the same indicators at a lower cost compared to the cost of the 'star' of development.